

Prueba de Caja Blanca

“Sistema de Gestión de Caja para Tags y Productos”

Integrantes:
Leopoldo Bolaños
Matias Medina
Martin Palacios

Fecha: 2026/06/19

REQ005 (Precio Variable):

Función que permite digitar manualmente conceptos y montos libres para consumos de papelería, validando rigurosamente que el valor ingresado sea un número decimal positivo.

1. CÓDIGO FUENTE para Precio Variable

- Controlador (Capturas)

```
1 package espe.edu.ec.programacaja.controller;
2
3 import espe.edu.ec.programacaja.model.CashFlowModel;
4 import espe.edu.ec.programacaja.view.VariableServicePanel;
5
6 import javax.swing.*;
7 import java.util.ArrayList;
8 import java.util.List;
9
10 public class VariableServiceController {
11
12     private VariableServicePanel view;
13     private CashFlowModel cashFlowModel;
14
15     private List<VariableServiceItem> temporalServices;
16
17     private static class VariableServiceItem {
18         private String concept;
19         private double amount;
20
21         public VariableServiceItem(String concept, double amount) {
22             this.concept = concept;
23             this.amount = amount;
24         }
25     }
26
27     public VariableServiceController(VariableServicePanel view, CashFlowModel cashFlowModel) {
28         this.view = view;
29         this.cashFlowModel = cashFlowModel;
30         this.temporalServices = new ArrayList<>();
31
32         initEvents();
33         actualizarResumenTemporal();
34     }
35
36     private void initEvents() {
```

```

35
36 private void initEvents() {
37     view.getBtnAgregar().addActionListener(e -> agregarServicioTemporal());
38
39     view.getBtnGuardarServicios().addActionListener(e -> guardarServiciosEnCaja());
40 }
41
42 private void agregarServicioTemporal() {
43     String concepto = view.getConcepto().trim();
44     String valorTexto = view.getValor().trim().replace(",",".");
45
46     if (concepto.isEmpty() || valorTexto.isEmpty()) {
47         showError("Debe ingresar el concepto y el valor del servicio.");
48         return;
49     }
50
51     double valor;
52
53     try {
54         valor = Double.parseDouble(valorTexto);
55     } catch (NumberFormatException ex) {
56         showError("El valor debe ser numérico. Ejemplo: 3.50");
57         return;
58     }
59
60     if (valor <= 0) {
61         showError("El valor debe ser mayor a cero.");
62         return;
63     }
64
65     temporalServices.add(new VariableServiceItem(concepto, valor));
66
67     view.limpiarCampos();
68     actualizarResumenTemporal();
69 }
70
71 private void guardarServiciosEnCaja() {
72     if (temporalServices.isEmpty()) {
73         showError("No hay servicios variables para guardar.");
74         return;
75     }
76
77     try {
78         for (VariableServiceItem item : temporalServices) {
79             cashFlowModel.addVariableServiceSale(item.concept, item.amount);
80         }
81
82         temporalServices.clear();
83         actualizarResumenTemporal();
84
85         JOptionPane.showMessageDialog(
86             view,
87             "Servicios variables guardados correctamente en el cierre de caja.",
88             "Servicios guardados",
89             JOptionPane.INFORMATION_MESSAGE
90         );
91     } catch (IllegalArgumentException ex) {
92         showError(ex.getMessage());
93     }
94 }
95
96 private void actualizarResumenTemporal() {
97     if (temporalServices.isEmpty()) {
98         view.setResumenTemporal("No hay servicios variables agregados.");
99         return;
100     }
101
102     StringBuilder sb = new StringBuilder();
103     double subtotal = 0.0;
104
105     sb.append("SERVICIOS VARIABLES TEMPORALES\n\n");
106

```

```

108         for (VariableServiceItem item : temporalServices) {
109             subtotal += item.amount;
110
111             sb.append(String.format(
112                 "%s = $%.2f\n",
113                 item.concept,
114                 item.amount
115             ));
116         }
117
118         sb.append(String.format("\nSubtotal temporal servicios variables: $%.2f", subtotal));
119
120         view.setResumenTemporal(sb.toString());
121     }
122
123     private void showError(String message) {
124         JOptionPane.showMessageDialog(
125             view,
126             message,
127             "Error",
128             JOptionPane.ERROR_MESSAGE
129         );
130     }
131 }

```

Controlador (Código)

```
package espe.edu.ec.programacaja.controller;
```

```
import espe.edu.ec.programacaja.model.CashFlowModel;
```

```
import espe.edu.ec.programacaja.view.VariableServicePanel;
```

```
import javax.swing.*;
```

```
import java.util.ArrayList;
```

```
import java.util.List;
```

```
public class VariableServiceController {
```

```
    private VariableServicePanel view;
```

```
    private CashFlowModel cashFlowModel;
```

```
    private List<VariableServiceItem> temporalServices;
```

```
    private static class VariableServiceItem {
```

```
        private String concept;
```

```
        private double amount;
```

```
        public VariableServiceItem(String concept, double amount) {
```

```
            this.concept = concept;
```

```
            this.amount = amount;
```

```
        }
```

```
    }
```

```
    public VariableServiceController(VariableServicePanel view, CashFlowModel cashFlowModel) {
```

```
        this.view = view;
```

```
        this.cashFlowModel = cashFlowModel;
```

```
        this.temporalServices = new ArrayList<>();
```

```
        initEvents();
```

```
        actualizarResumenTemporal();
```

```
    }
```

```

private void initEvents() {
    view.getBtnAgregar().addActionListener(e -> agregarServicioTemporal());

    view.getBtnGuardarServicios().addActionListener(e -> guardarServiciosEnCaja());
}

private void agregarServicioTemporal() {
    String concepto = view.getConcepto().trim();
    String valorTexto = view.getValor().trim().replace(",", ".");

    if (concepto.isEmpty() || valorTexto.isEmpty()) {
        showError("Debe ingresar el concepto y el valor del servicio.");
        return;
    }

    double valor;

    try {
        valor = Double.parseDouble(valorTexto);
    } catch (NumberFormatException ex) {
        showError("El valor debe ser numérico. Ejemplo: 3.50");
        return;
    }

    if (valor <= 0) {
        showError("El valor debe ser mayor a cero.");
        return;
    }

    temporalServices.add(new VariableServiceItem(concepto, valor));

    view.limpiarCampos();
    actualizarResumenTemporal();
}

private void guardarServiciosEnCaja() {
    if (temporalServices.isEmpty()) {
        showError("No hay servicios variables para guardar.");
        return;
    }

    try {
        for (VariableServiceItem item : temporalServices) {
            cashFlowModel.addVariableServiceSale(item.concept, item.amount);
        }

        temporalServices.clear();
        actualizarResumenTemporal();

        JOptionPane.showMessageDialog(

```

```

        view,
        "Servicios variables guardados correctamente en el cierre de caja.",
        "Servicios guardados",
        JOptionPane.INFORMATION_MESSAGE
    );

} catch (IllegalArgumentException ex) {
    showError(ex.getMessage());
}
}

private void actualizarResumenTemporal() {
    if (temporalServices.isEmpty()) {
        view.setResumenTemporal("No hay servicios variables agregados.");
        return;
    }

    StringBuilder sb = new StringBuilder();
    double subtotal = 0.0;

    sb.append("SERVICIOS VARIABLES TEMPORALES\n\n");

    for (VariableServiceItem item : temporalServices) {
        subtotal += item.amount;

        sb.append(String.format(
            "%s = $%.2f\n",
            item.concept,
            item.amount
        ));
    }

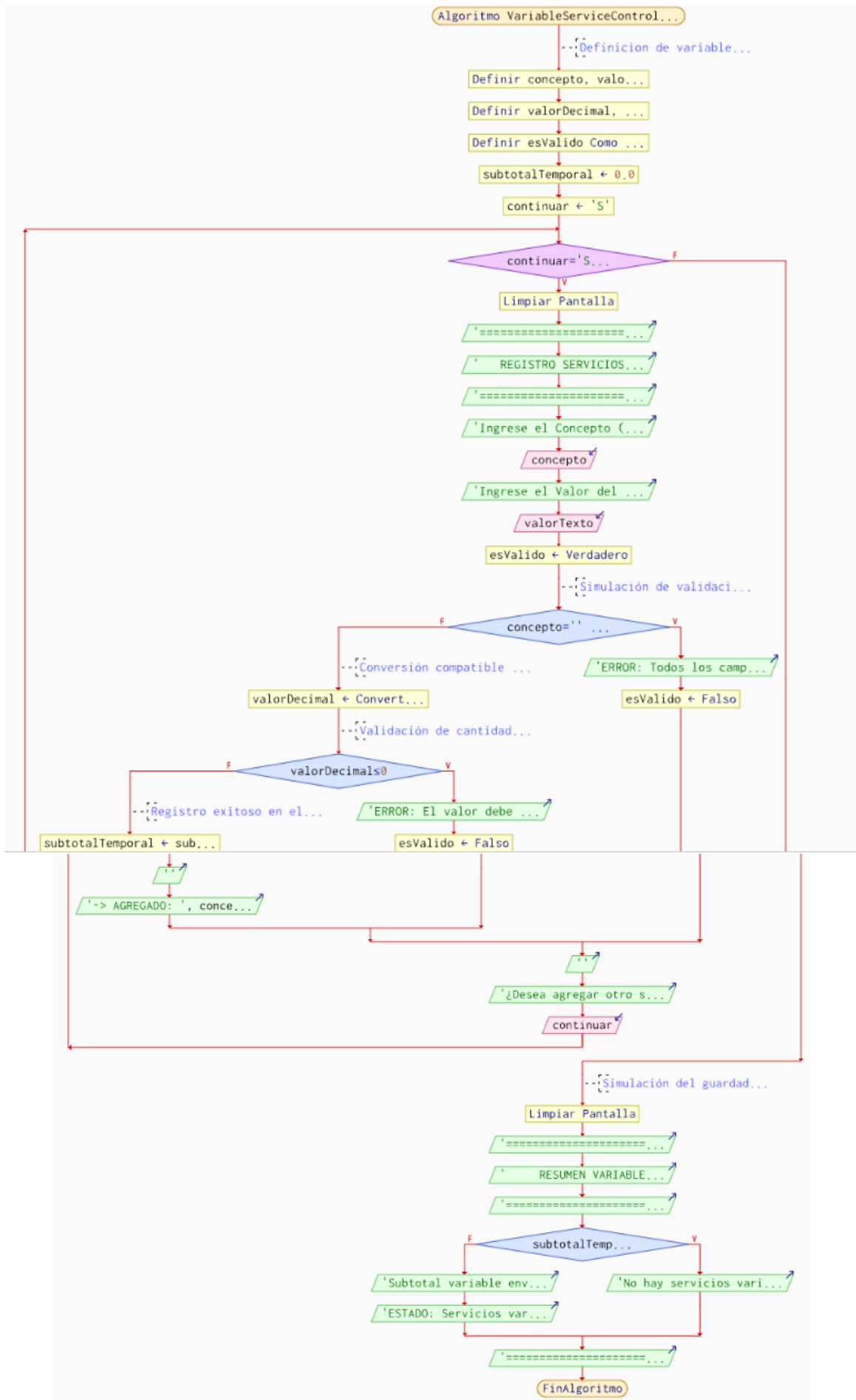
    sb.append(String.format("\nSubtotal temporal servicios variables: $%.2f", subtotal));

    view.setResumenTemporal(sb.toString());
}

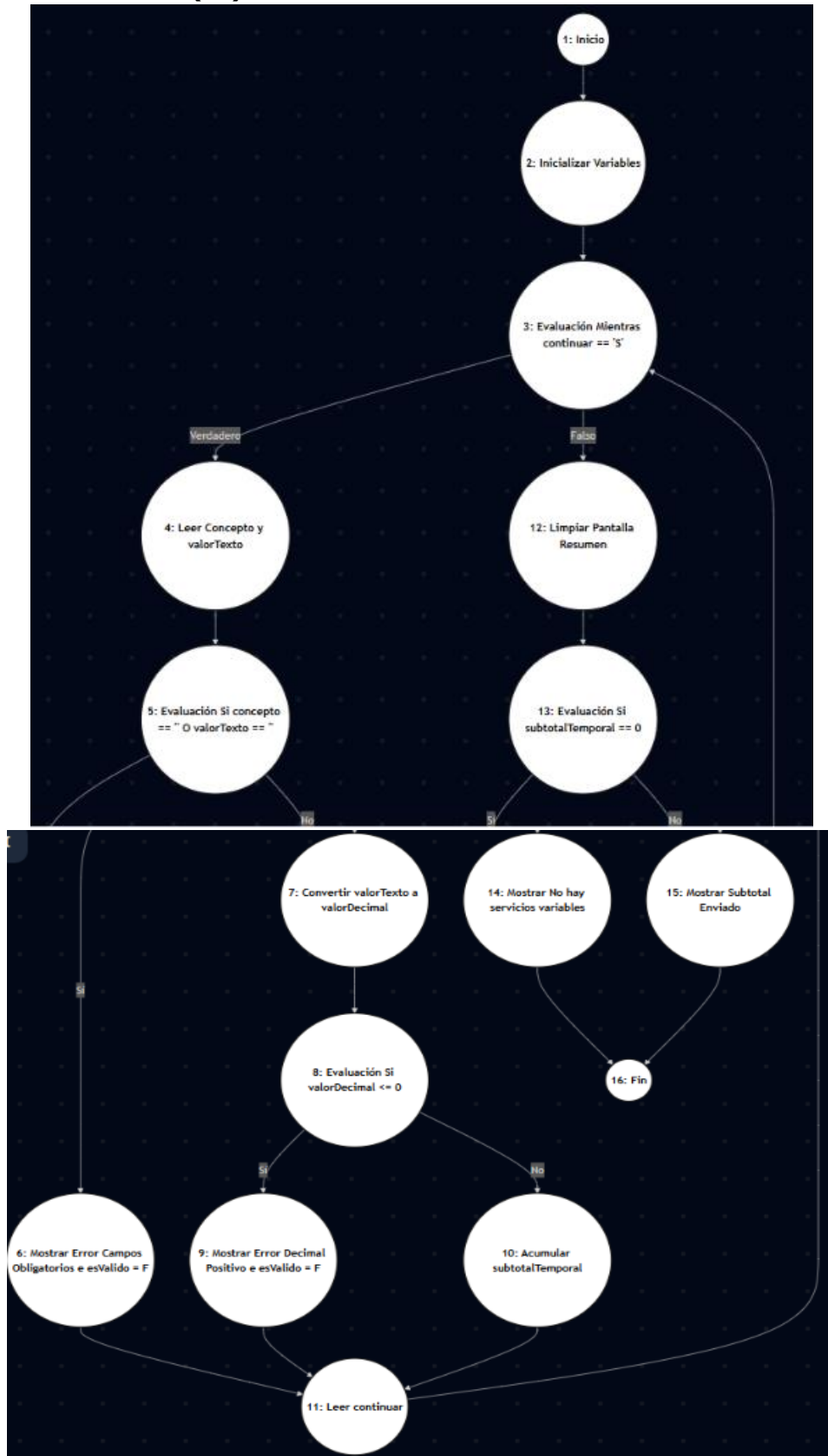
private void showError(String message) {
    JOptionPane.showMessageDialog(
        view,
        message,
        "Error",
        JOptionPane.ERROR_MESSAGE
    );
}
}

```

2. DIAGRAMA DE FLUJO (DF) Precio Variable



3. GRAFO DE FLUJO (GF) Precio Variable



4. IDENTIFICACIÓN DE LAS RUTAS (Camino básico) Precio Variable

Determinar en base al GF del numeral 4

RUTAS

R1 (Camino de Salida Directa / Sin Ventas Guardadas): 1 - 2 - 3 [Falso] - 12 - 13 [Sí] - 14 - 16

R2 (Validación de Campos Vacíos): 1 - 2 - 3 [Verdadero] - 4 - 5 [Sí] - 6 - 11 - 3 [Falso] - 12 - 13 [Sí] - 14 - 16

R3 (Validación de Valor Negativo o Cero): 1 - 2 - 3 [Verdadero] - 4 - 5 [No] - 7 - 8 [Sí] - 9 - 11 - 3 [Falso] - 12 - 13 [Sí] - 14 - 16

R4 (Flujo Exitoso con Registro Acumulado): 1 - 2 - 3 [Verdadero] - 4 - 5 [No] - 7 - 8 [No] - 10 - 11 - 3 [Falso] - 12 - 13 [No] - 15 - 16

5. COMPLEJIDAD CICLOMÁTICA Precio Variable

Se puede calcular de las siguientes formas:

V/G) Nodos predicados (IF-THEN-ELSE) +1

$V(G) = 4 + 1$

$V(G) = 5$

$V(G) = A - N + 2$

$V(G) = 19 - 16 + 2$

$V(G) = 5$

DONDE:

P: Número de nodos predicado

A: Número de aristas

N: Número de nodos